

Build with LLMs: Open Source AI for the JVM

Wednesday, September 16 2026, 1-2 PM PST

Silicon Valley Campus, Mountain View, CA, USA

2026

Microsoft

>hackathon_

SPEAKER

>h_

MICROSOFT



KANNUPRIYA KALRA



BUILD OPEN. MOVE FAST. SHIP 1.0.

The Journey to LLM4S 1.0



LIVE DEMOS



ARCHITECTURE



LESSONS LEARNED



Microsoft already runs on the JVM

500K+

JVMs deployed inside Microsoft

Not counting Azure services or customer workloads

Source: Microsoft for Java Developers blog, Microsoft Build of OpenJDK announcement (2021)



Microsoft Build of OpenJDK

Microsoft's own free, open-source LTS OpenJDK distribution, generally available since 2021.



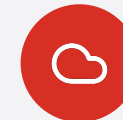
Scala, Kotlin and Java

Microsoft's Java engineering team has said its internal JVM systems are built in all three.



MCP, co-shaped by Microsoft

Microsoft and GitHub joined the MCP Steering Committee at Build 2025. LLM4S agents speak MCP.



Azure OpenAI, first class

Azure OpenAI is a fully supported LLM4S provider, on the same code path as every other model.

Today's journey

01

The real AI problem

Why demos work and systems don't

02

Why Scala and the JVM

Types, errors as values, and the runtime you already run

03

What LLM4S does

Architecture, capabilities and live demos

04

What worked, what failed

Lessons from building in public since early 2025

05

Build it this week

Hackathon quick start and project ideas

Demos amaze. Systems collapse under real load.

95%

**of enterprise GenAI pilots show
no measurable business return**

Source: MIT NANDA, The GenAI Divide: State of AI in Business 2025

Why pilots stall



Ungrounded answers

Models hallucinate when nothing ties them to real data.



Latency and cost at scale

What feels instant in a demo gets slow and expensive under real traffic.

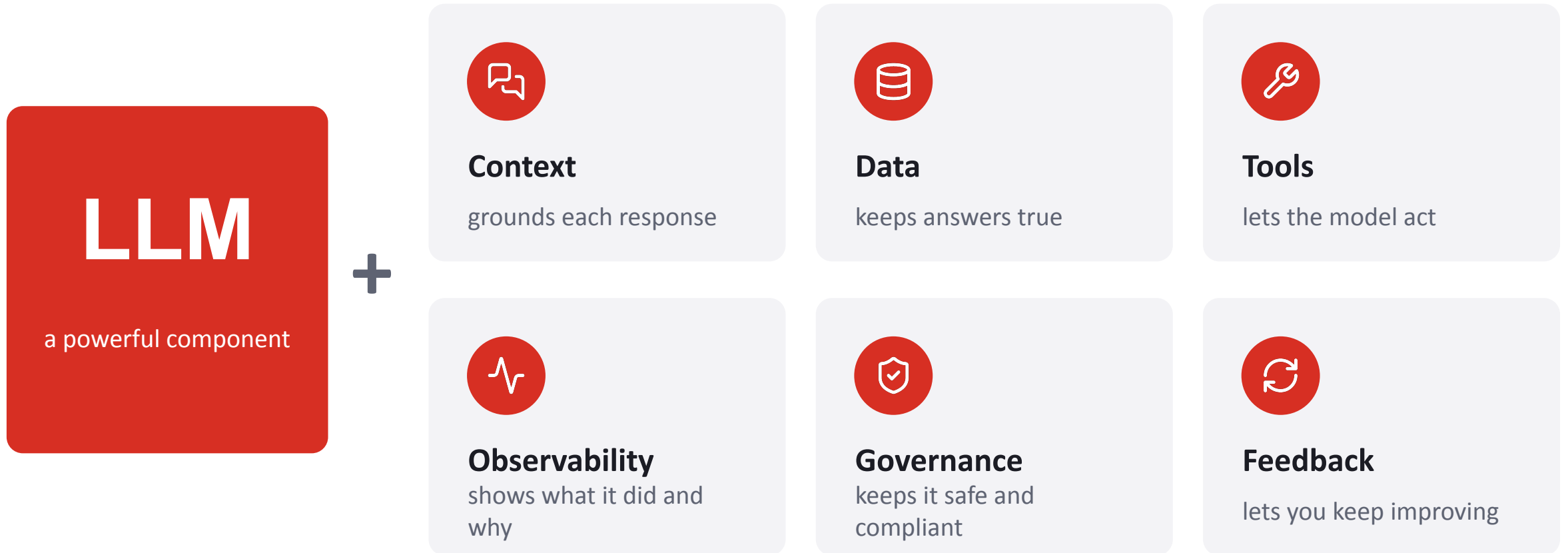


No way to see or measure

No traces, no metrics, no feedback loop, so no way to improve.

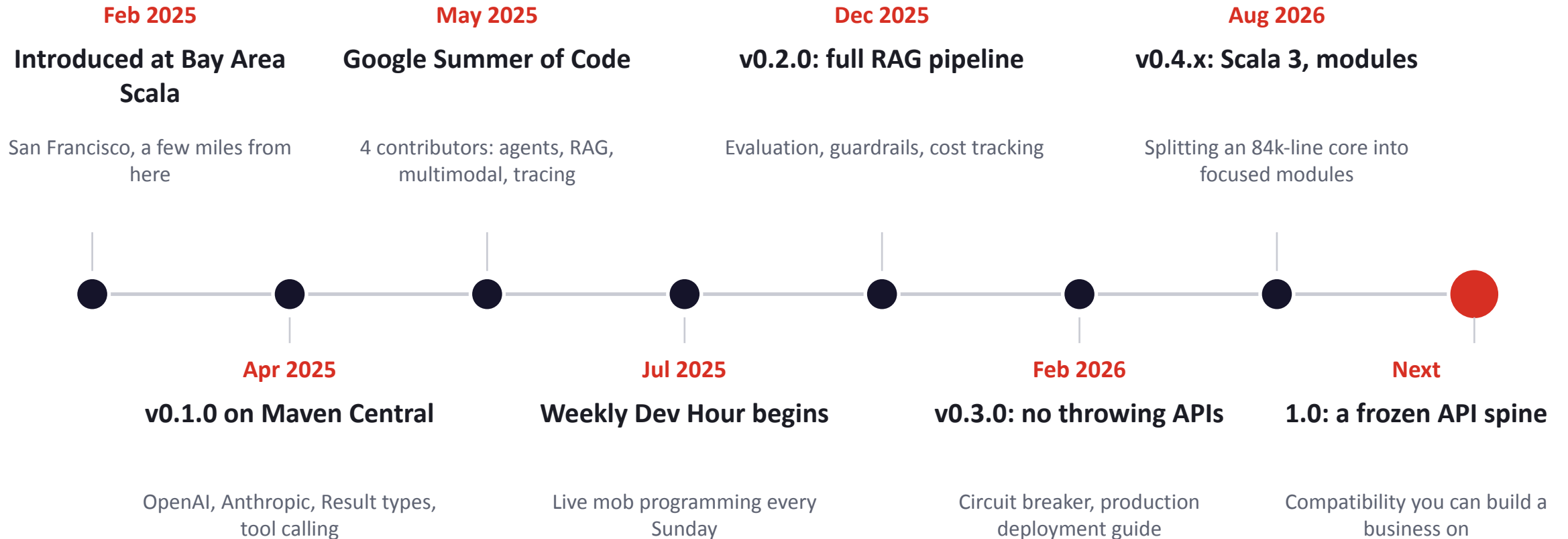
The shift: from cool prototype to critical system.

An LLM is not a product



LLM4S gives every one of these layers a typed, testable home.

From idea to 1.0, in public



Why Scala for LLM applications?

Most LLM work happens in Python.

We believe production AI deserves a stronger foundation: one where mistakes surface at compile time, not at 2am.



Types turn runtime surprises into compile errors

Message roles, config, tool schemas and provider cases are checked before you ship.



Errors are values

Result[T] makes every failure path explicit and composable.



Immutable conversations

State you can persist, resume, test and reason about.



The JVM you already run

Mature libraries, real concurrency, deployed next to your services.

The typo that compiled and shipped

✗ Before: roles were strings

```
// role is just a String
val msg = Message(
  role    = "asistant", // typo
  content = reply
)

// compiles, passes review,
// misbehaves in production
```

✓ After: roles are an enum

```
// a closed set of cases
enum MessageRole:
  case System, User, Assistant, Tool

MessageRole.Asistant
// error: value Asistant is not
// a member of MessageRole
```

43

files migrated from strings to types

6

type classes introduced

0

ways left to misspell a role

Errors are values, not surprises

```
val result = for {  
  config  <- Llm4sConfig.provider()  
  client  <- LLMConnect.getClient(config)  
  response <- client.complete(  
    messages = List(UserMessage("Explain MCP")),  
    model     = None  
  )  
} yield response  
  
result match {  
  case Right(c) => println(c.content)  
  case Left(err) => println(s"Error: $err")  
}
```

47

try-catch blocks deleted in one safety refactor

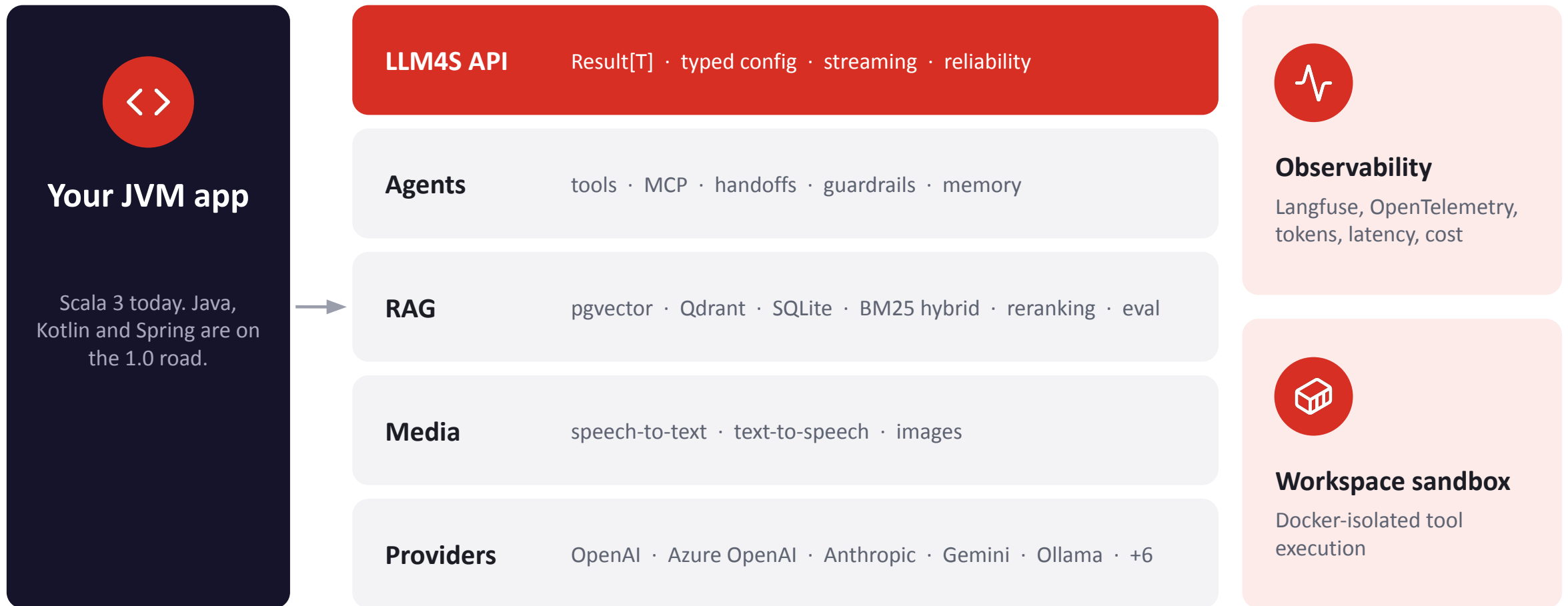
0

throwing APIs left after v0.3.0

1

typed error hierarchy, replacing "Unknown error" for everything

How LLM4S fits together



What you can build with today



Multi-provider

11 providers behind one API.
Switch with config.



Tool calling + MCP

Typed tools with generated
schemas, local or over MCP.



Agent orchestration

Multi-step agents, handoffs
to specialists, guardrails.



Memory

Short and long-term memory
with entity tracking.



Observability

Tracing, token usage, latency
and cost.



Streaming

Real-time tokens and agent
events, with backpressure.



Speech + multimodal

Speech-to-text,
text-to-speech, image
generation.



Production patterns

Retries, circuit breakers,
sandboxed tools, deploy
guide.

One API, eleven providers

```
# pick a model with one line of config
export LLM_MODEL=openai/gpt-4o
export OPENAI_API_KEY=sk-...

# same code, different provider
export LLM_MODEL=anthropic/claude-sonnet-4-5-latest
export ANTHROPIC_API_KEY=...

# Azure OpenAI
export AZURE_API_BASE=...
export AZURE_API_KEY=...
```

Same application code. Change config, not business logic.

Supported today

OpenAI

Azure OpenAI

Anthropic

Google Gemini

Ollama (local)

DeepSeek

Mistral

Cohere

OpenRouter

Requesty

Z.ai

Frozen at 1.0: OpenAI and compatible clients, Anthropic, Gemini, Ollama

Letting the model act, safely

```
/** Look up an order by its id
 * @param id the order to find
 */
def lookupOrder(id: String): ToolResponse =
  orders.find(id)

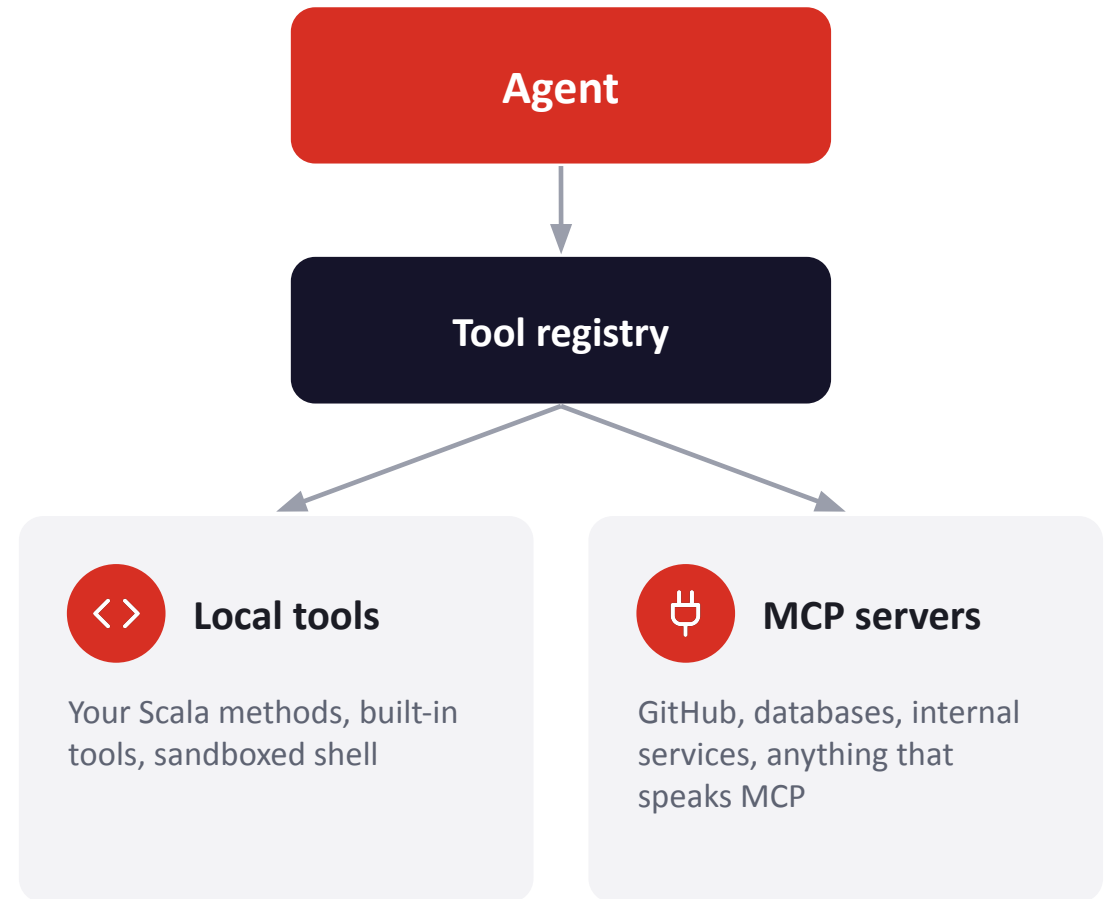
// LLM4S reads the name, types and docs
// and generates the tool schema for you
```



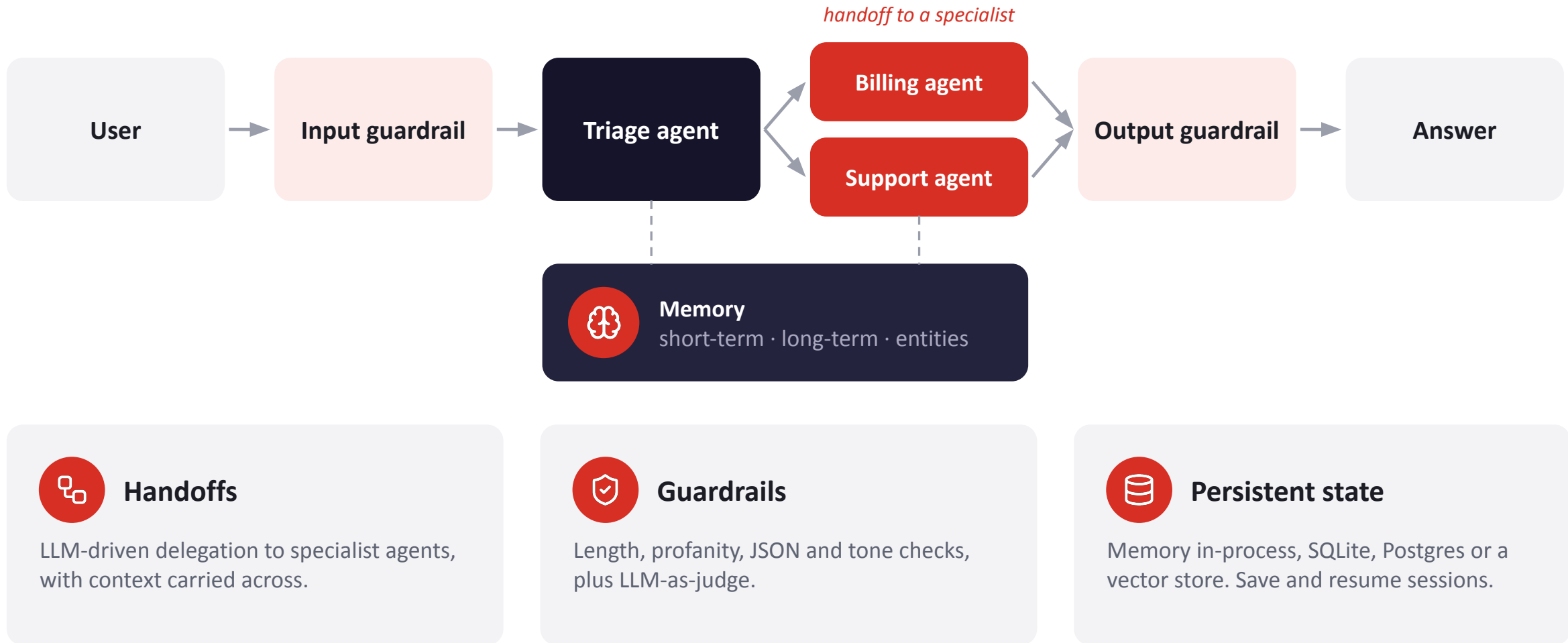
Built-in tools: date and time, calculator, HTTP, web search, files



Tools can run in a Docker-isolated workspace



Agents that hand off, check and remember



See every step, as it happens

Trace: support-agent run

2.4 s · 1,912 tokens · \$0.011

agent.run

guardrail.input

llm.complete

tool.lookupOrder

llm.complete

guardrail.output

Illustrative trace. Real traces go to Langfuse or your console.



Tracing

- Langfuse or console, set by TRACING_MODE
- OpenTelemetry export module
- Tokens, latency and cost per call
- Queryable in-process TraceStore



Streaming

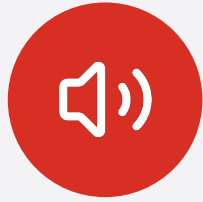
- Token streaming with backpressure
- Agent event streams for live UIs
- Raw provider exchange logs for debugging

Beyond text: voice and images



Speech-to-text

Turn voice into input for the same agents and tools.



Text-to-speech

Give your app a voice without leaving the JVM.



Images

Generate and analyze images through one API.



Honest labels: speech and image APIs are marked Experimental for 1.0. They work today, and they may still change. We'd rather tell you now than surprise you later.



LET'S BUILD

Live demo

01

One app, many providers

Switch models by changing config

02

An agent with tools

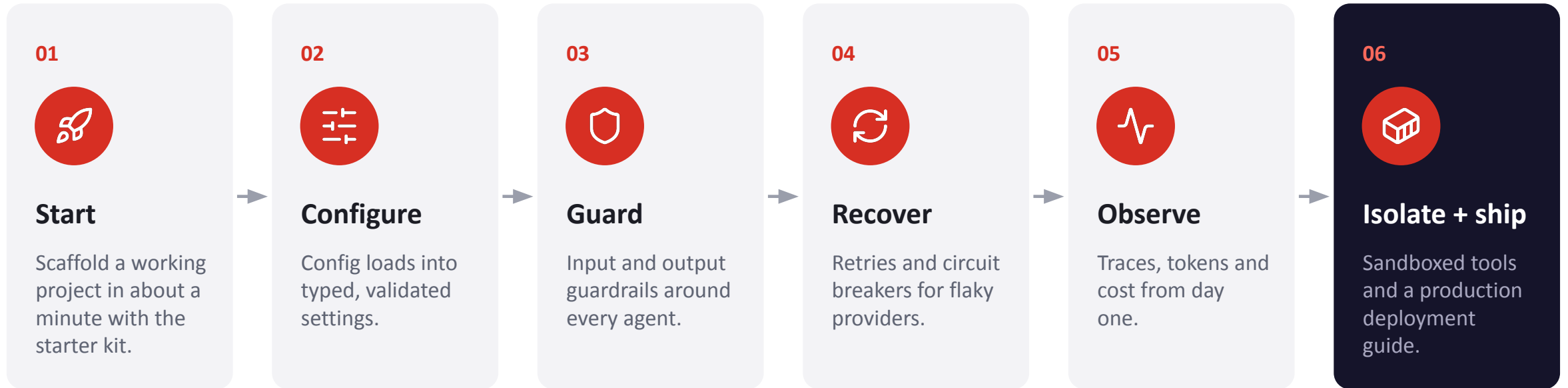
Calling local tools and an MCP server

03

Trace the run

Tokens, latency and cost for every step

From experiment to production



Rule of thumb: if you can't see it, test it and roll it back, it isn't production yet.

What worked, what failed, what we learned



What worked

- Types over strings: whole classes of bugs gone
- Result[T] everywhere instead of exceptions
- A starter kit: setup from 20 minutes to 60 seconds
- Building in public: weekly live coding and GSoC mentoring



What failed

- One 84k-line core artifact: every user got everything
- Unused JDBC drivers and parsers leaking onto users' classpaths
- "Unknown error" for every failure, early on
- Breadth first: provider parity and contract tests came later



What we learned

- Freeze a small spine, let the edges move
- Module boundaries follow the dependency graph
- Label maturity honestly: Frozen, Beta, Experimental
- Say no to scope: 1.0 targets Scala 3 only

What 1.0 promises

Frozen at 1.0

Source and binary compatible across 1.x. Deprecate before removing.

llm4s-core

llm4s-agent

llm4s-observability

OpenAI + compatible clients

Anthropic, Gemini, Ollama

Beta

Usable now, still hardening. Migration notes ship with every change.

RAG and vector stores

Memory (+ Postgres)

MCP

Shared media types

Community providers

Experimental

Useful and evolving. Expect the API to change.

Speech

Image generation + processing

Knowledge graph

Today: v0.4.1 is released, and the module split is in progress on main. Binary compatibility is enforced with MiMa checks.

Build with LLM4S this week

Get running in three steps

1 Create a project

```
sbt new llm4s/llm4s.g8
```

2 Point it at a model

```
export LLM_MODEL=openai/gpt-4o
```

3 Or add it to an existing build

```
"org.llm4s" %% "llm4s-core" % "0.4.1"
```

Works with Azure OpenAI, or fully local with Ollama.

Project ideas



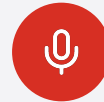
Team wiki Q&A

RAG over your docs with pgvector and reranking



MCP ops agent

Reads issues, calls tools, drafts a fix



Voice front door

Speech in, agent in the middle, speech out



Support bot

Triage agent, handoffs and output guardrails

Stuck? Find us on the LLM4S Discord.

The road ahead, and how to join

On the road to 1.0

- Java facade and Kotlin coroutine API
- Spring Boot starter, Maven and Gradle examples
- Provider capability matrix and contract tests
- Threat model, benchmarks and release gates
- Maintained reference applications

Join the community



Discord

Questions, design discussions, help



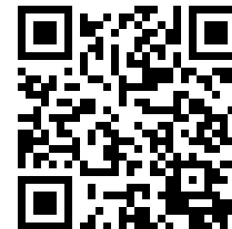
LLM4S Dev Hour

Live coding every Sunday, 9am London



Mentorship

Ideas for GSoC, LFX and European Summer of Code



github.com/llm4s/llm4s

A star helps more than you think.

KEY TAKEAWAYS

Build the cool stuff, and make it work in production.

- 01 Reliable AI is designed in, not tested in at the end.
- 02 Frameworks beat ad-hoc scripts once you have real users.
- 03 Types and Result values make LLM apps predictable.
- 04 LLM4S brings this to the JVM you already run.



Kannupriya Kalra

Engineering Leader | AI | Scala Back-end |
Google Summer Of Code 2024 & 2025 Org Ad...



Connect on LinkedIn

Questions?

kannupriya@llm4s.org · rory@llm4s.org
llm4s.org · github.com/llm4s

Thank you!

Star us on GitHub, then come build with us.



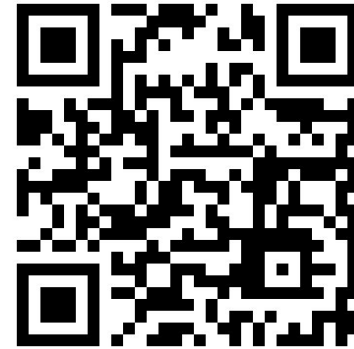
GitHub

github.com/llm4s/llm4s



Docs

llm4s.org



Discord

[Join the community](#)